Departmental Inventory Checker

Project overview

The Departmental Inventory Checker is a MERN-stack web application for tracking a college department's laboratory equipment, locations, stock, borrowing activity, and maintenance records.

It provides a shared system for staff to check what equipment is recorded, where it is assigned, how much is available, what has been issued, and which items need attention.

Main features

• **Equipment:** asset codes, categories, lab locations, quantities, condition, and stock thresholds.

• **Laboratories:** room details, department, capacity, and equipment assignment.

• **Transactions:** request, approval, issue, and return tracking.

• **Maintenance:** issue type, priority, status, and resolution information.

• **Reports:** dashboard metrics and CSV exports for inventory and maintenance.

User roles

Role	Access
Administrator	Manages users, inventory, laboratories, transactions, maintenance, and reports.
Lab assistant	Supports inventory and laboratory operations, issue/return handling, and maintenance tracking.
Faculty	Views inventory, requests equipment, and reports equipment issues. Public signup creates faculty accounts only.

Project context

This project demonstrates how a department can replace disconnected stock and checkout records with a role-based application backed by a database. Included sample records are for development and demonstration.

Workflow and technical design

Equipment issue and return

• Faculty submits an equipment request.

• An administrator or lab assistant reviews and approves it.

• Issuing equipment reduces available stock.

• Recording a return increases available stock and marks the transaction returned.

The server checks stock availability and updates the equipment and transaction records during issue and return operations.

Technology

Frontend: React, Vite, React Router, Tailwind CSS, and Axios.

Backend: Node.js and Express.

Database: MongoDB with Mongoose models for users, laboratories, equipment, transactions, and maintenance records.

Authentication: JWT and bcryptjs.

Security

Passwords are hashed with bcrypt. Authentication uses an httpOnly cookie. Backend role checks protect restricted operations. Faculty signup cannot create administrator or lab-assistant accounts.

Run locally

• Install Node.js and npm, and make MongoDB available.

• Run `npm install` from the project root.

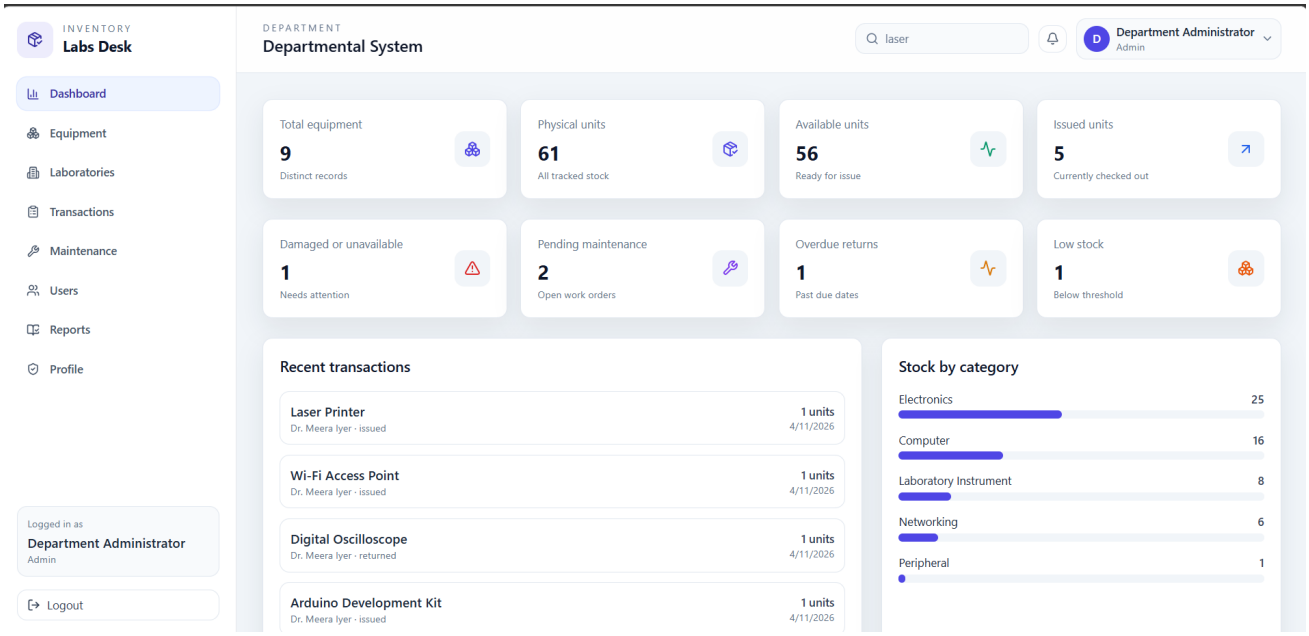
• Copy the example environment files to `server/.env` and `client/.env`. Configure MongoDB, a unique private JWT secret, and the API URL.

• Set the administrator bootstrap values and run `npm run bootstrap:admin`.

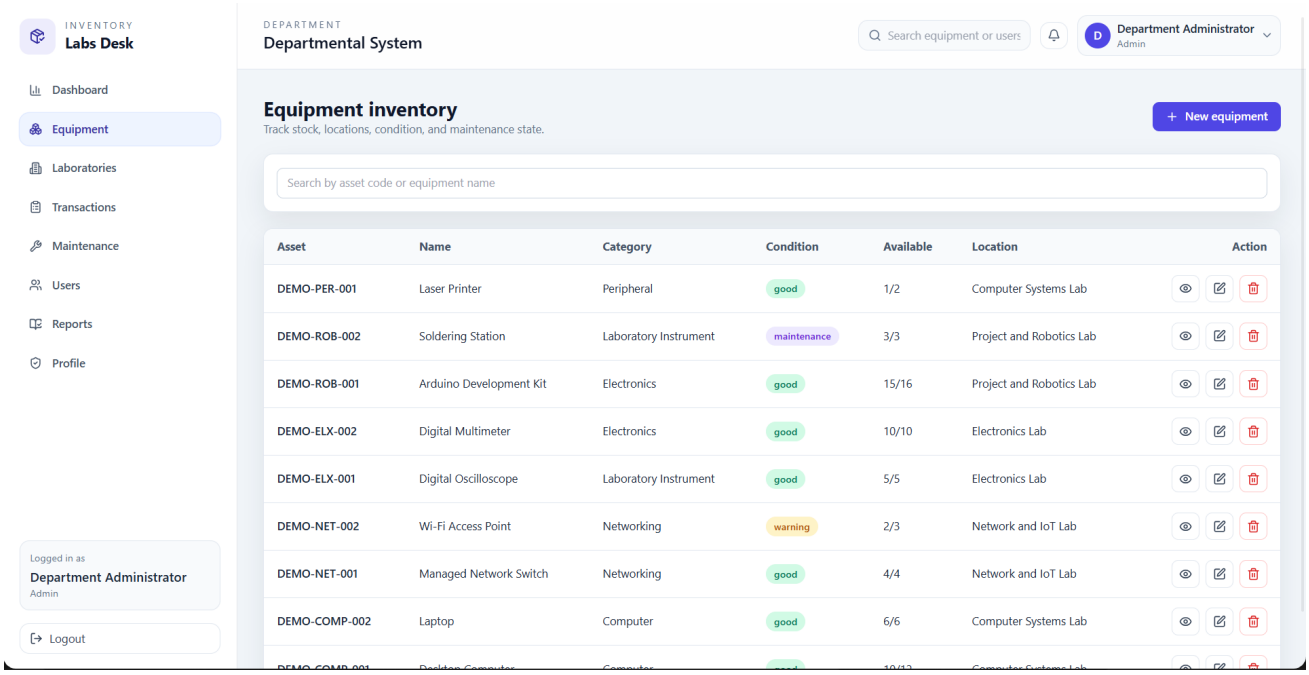
• Optionally run `npm run seed:demo`, then start the application with `npm run dev`.

The project provides `npm test` for server tests covering authentication, stock validation, equipment issuance, and maintenance resolution. During project validation, the client build passed and nine backend tests passed.

Application screenshots

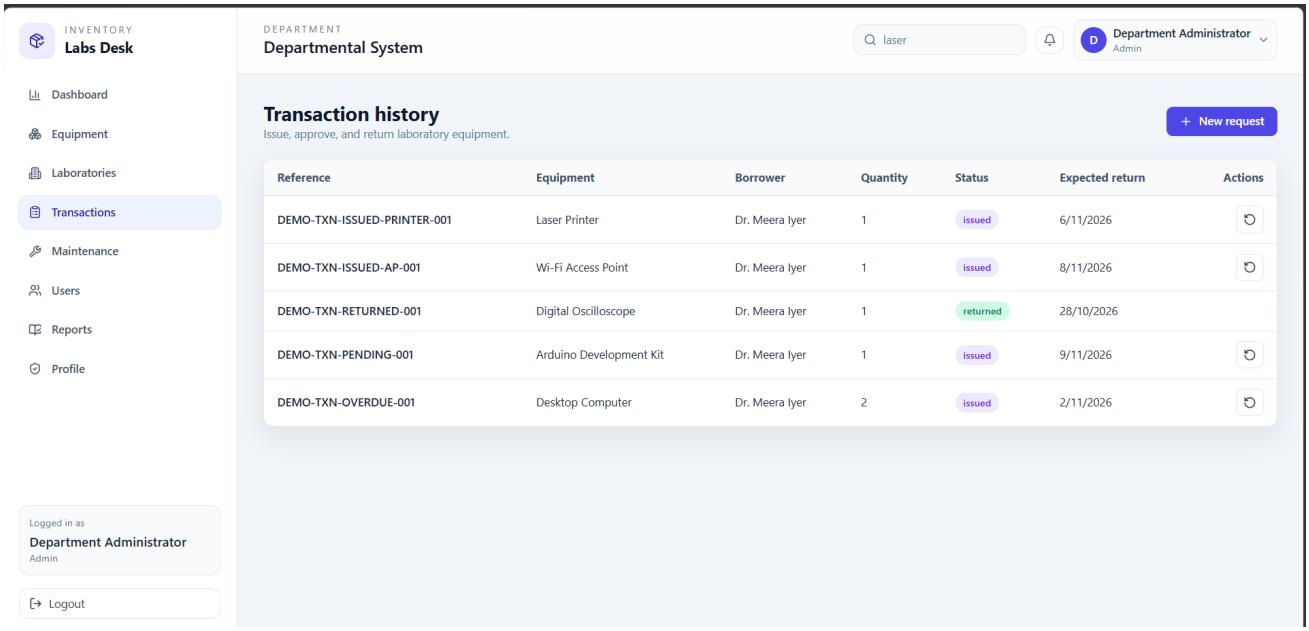


Dashboard: inventory totals, availability, issued items, and recent activity.

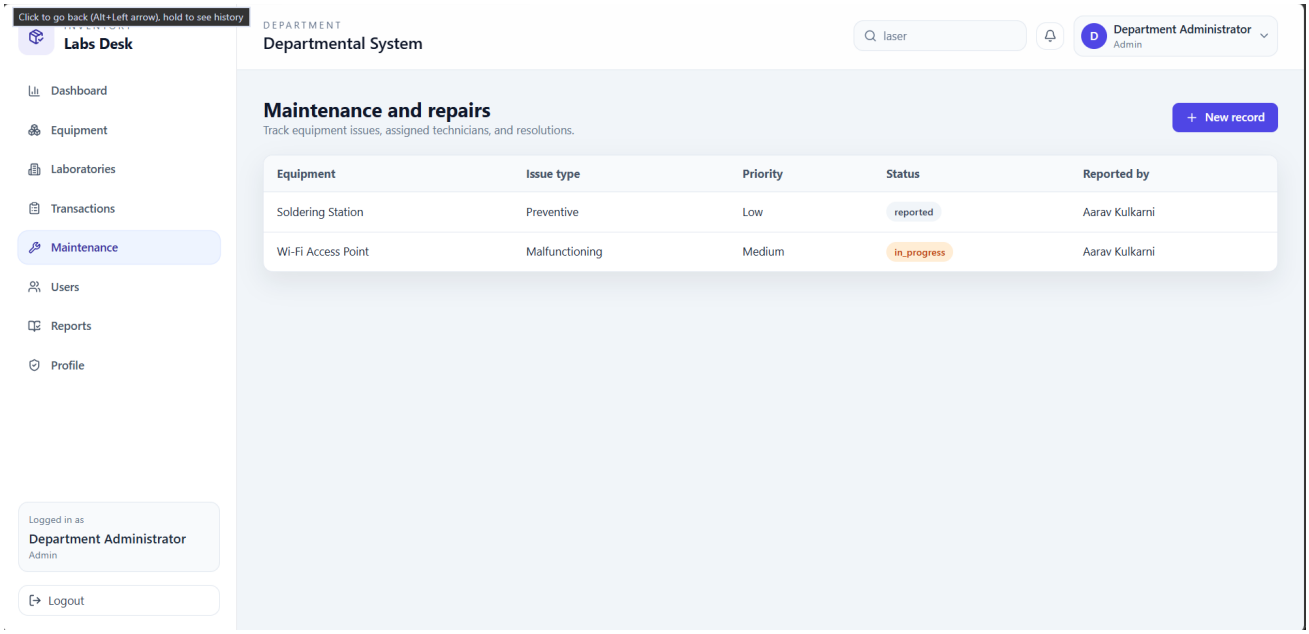


Equipment inventory: asset records, condition, stock, and laboratory location.

Application screenshots

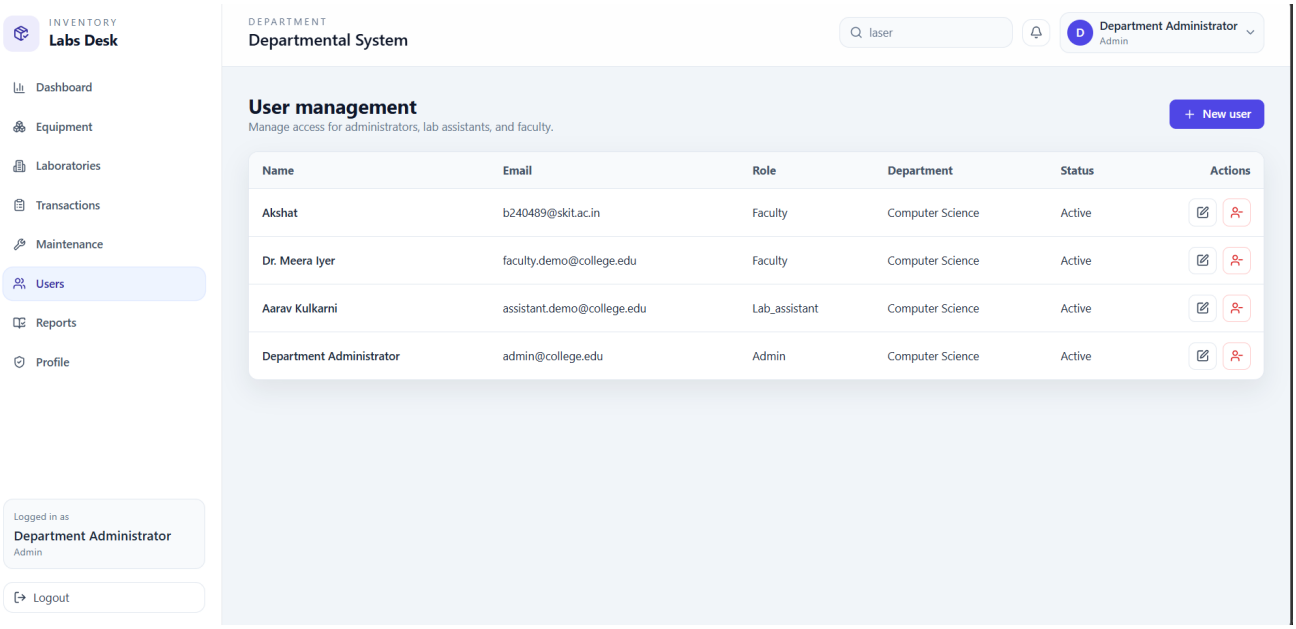


Transactions: equipment, borrower, quantity, status, and expected return.



Maintenance: equipment issue, priority, status, and reporter.

Application screenshot



User management: active accounts, roles, and departments.